# Requirements Gathering & Specification Document

**Project Title:** Real-Time Cold Chain Fleet Telemetry Pipeline

**Group Code:** DKH5_AIS5_S1

**Track:** Data Engineering (Round 5)

**Company:** Skills Dynamix

**Project Supervisor:** Eng. Abdelrahman Mousa

**Team Members:** Youssef Elsayed (Leader), Zakaria Ahmed, Abdalaziz Ayman, Abdelmonem Khaled, David Fady, Muhammad Ali

## 1. Identify Stakeholders

### Stakeholder Identification & Categorization

- **Internal Stakeholders:**
    - **Junior Data Engineering Team:** Responsible for system design, data modeling, pipeline engineering, and end-to-end integration.
    - **Project Supervisor (Eng. Abdelrahman Mousa):** Evaluates architectural integrity, engineering best practices, milestone deliverables, and code quality.
- **External & Business Stakeholders:**
    - **Fleet Logistics Managers:** Require unified real-time visibility over fleet locations, transit routes, and delay estimations to optimize dispatching.
    - **Cold Chain Quality Assurance Officers:** Require continuous monitoring of cargo environmental parameters (refrigeration temperatures) to prevent cargo spoilage and satisfy compliance regulations.
    - **Receiving Customers / Consignees (Automotive & Industrial Clients):** Demand on-time delivery guarantees, accurate ETAs, and immediate notifications regarding transit disruptions.

## Stakeholder Influence & Interest Matrix

| Stakeholder Group | Interest Level | Influence/Power | Primary Alignment Focus |
|---|---|---|---|
| **Fleet Operations Managers** | High | High | Actionable real-time dashboards and delay indicators |
| **Cold Chain Safety Officers** | High | High | Sub-second anomaly alerts on temperature violations |
| **Project Supervisor** | High | High | Robust pipeline architecture, code modularity, and containerization |
| **Data Engineering Team** | High | Medium | Feasible technical execution, clear task boundaries, and clean schema |
| **End Customers (Consignees)** | Medium | Low | Delivery accuracy and verified transit conditions |

**2. Plan the Approach**

**Project Objectives & High-Level Scope**

The primary objective is to build a containerized, end-to-end real-time data streaming pipeline capable of ingesting simulated IoT telematics from a distributed fleet, validating and processing the events in-stream, storing the time-series records in an optimized database, and surfacing operational insights via an interactive dashboard.

**Timeline & Milestones**

- **Duration:** 4 Weeks structured around agile weekly sprints.
- **Milestone 1 (Week 1):** Infrastructure containerization and dataset profiling.
- **Milestone 2 (Week 2):** IoT telematics streaming and time-series database modeling.
- **Milestone 3 (Week 3):** Stream transformation, data enrichment, and persistence.
- **Milestone 4 (Week 4):** Dashboard creation, pipeline stress testing, and final sign-off.

**Requirements Elicitation Techniques**

1. **Data Profiling & Source Analysis:** Analyzing the *Transportation and Logistics Tracking Dataset* (sheets: Primary data and Refined) to determine schema attributes, null rates, and streaming frequencies.

2. **Operational Scenario Modeling:** Mapping out realistic cold chain transit scenarios (e.g., normal transit vs. sudden cooling system failure).
3. **Weekly Supervisor Alignments:** Sprint review meetings to validate architecture decisions and technical trade-offs.

**3. Conduct Elicitation**

**Core Problem Definition**

Traditional fleet monitoring relies on batch-processed GPS logs and static reports generated after trip completion. In cold chain logistics, post-trip reporting fails to prevent spoilage of temperature-sensitive pharmaceuticals or food supplies. Furthermore, conventional relational databases experience severe write degradation under high-frequency IoT sensor ingestion.

**Extracted Business & Operational Needs**

- **Sub-Second Telemetry Ingestion:** The system must capture vehicle coordinates, velocity, and refrigeration metrics continuously.
- **In-Flight Anomaly Detection:** Immediate alert triggers when cargo temperature breaches the safety threshold ($> 8°C$).
- **Route Deviation & Delay Detection:** Continuous comparison between real-time location pings and scheduled arrival times (Planned_ETA).
- **High-Throughput Time-Series Persistence:** A dedicated storage engine capable of high ingestion rates without indexing bottlenecks.

**4. Document Observations and Findings**

**A. Functional Requirements (FR)**

- **FR-01 (IoT Telemetry Simulation):** The simulator must parse the logistics dataset and publish serialized JSON messages mimicking IoT sensors at configurable intervals (1–2 seconds).
- **FR-02 (Event Ingestion):** The messaging layer (MQTT Broker) must accept pub/sub connections from multiple simulated vehicle publishers with QoS 1 message delivery guarantees.

- **FR-03 (Stream Ingestion & Deserialization):** The streaming consumer must subscribe to the broker topics, deserialize incoming payloads, and validate data types.
- **FR-04 (Data Enrichment & Transformation):** The pipeline must clean incoming coordinates, calculate elapsed transit durations, compute delay indicators, and enrich records with operational attributes (weather conditions, cost metrics).
- **FR-05 (Time-Series Data Storage):** The system must persist enriched telemetry records into a TimescaleDB hypertable partitioned by event timestamp.
- **FR-06 (Alerting Mechanisms):** The pipeline must flag critical operational thresholds (e.g., temperature excursions $> 8°C$ or prolonged unplanned stoppages).
- **FR-07 (Interactive Live Dashboard):** The visualization layer (Grafana) must render real-time geospatial maps plotting active vehicle coordinates alongside time-series graphs of speed and temperature trends.

**B. Non-Functional Requirements (NFR)**

- **NFR-01 (Latency):** Total end-to-end processing latency from message publication to database persistence must remain below **1.5 seconds**.
- **NFR-02 (Reliability & Fault Tolerance):** The consumer application must implement automated reconnection logic in the event of message broker disconnects.
- **NFR-03 (Throughput & Scalability):** The ingestion pipeline must comfortably support concurrent streaming from up to 50 simulated vehicle nodes without message drop or buffer overflow.
- **NFR-04 (Data Integrity):** The database ingestion logic must discard malformed or corrupted payloads while recording rejected items in an error log.
- **NFR-05 (Maintainability & Modularity):** All pipeline components (simulator, consumer, database scripts) must be decoupled, modular, and configurable via environment variables (.env).
- **NFR-06 (Portability):** The entire infrastructure (broker, database, dashboard) must be deployable via a single docker-compose up command.

**C. Project Constraints (CON)**

- **CON-01 (Hardware Limitation):** No physical IoT sensor hardware; all streams must be simulated using historical logistics datasets.
- **CON-02 (Resource Footprint):** The deployment must operate smoothly on standard developer machines (16 GB RAM, with Docker memory limits capped at $\leq$ 6 GB).
- **CON-03 (Technology Stack):** Solutions must strictly rely on open-source technologies (Python, Mosquitto MQTT, PostgreSQL/TimescaleDB, Grafana, Docker).
- **CON-04 (Delivery Timeline):** Strict academic deadline of 4 weeks.

**5. Analyze and Prioritize (MoSCoW Framework)**

| Category | Requirement Code | Description |
|---|---|---|
| **Must Have (M)** | **FR-01, FR-02, FR-03, FR-05, NFR-06** | Python telemetry simulator, MQTT broker, consumer ingestion engine, TimescaleDB hypertable storage, and Docker Compose orchestration. |
| **Should Have (S)** | **FR-04, FR-06, FR-07, NFR-01** | Data enrichment (weather/costs), sub-second latency, temperature anomaly threshold alerts, and real-time Grafana geospatial mapping. |
| **Could Have (C)** | **Extended Features** | Apache Kafka integration for partitioned multi-topic benchmarks, or automated Webhook notifications (Slack/Telegram) on critical alerts. |
| **Won't Have (W)** | **Out of Current Scope** | Deployment of physical GPS/OBD hardware, driver mobile applications, or multi-region production cloud Kubernetes deployment. |

**6. Verify and Validate**

**Verification Mechanisms**

- **Automated Data Validation:** SQL consistency checks running against TimescaleDB to verify that incoming row counts match simulator event logs.
- **Dry-Run End-to-End Walkthrough:** A complete integration test demonstrating a simulated vehicle emitting pings that appear on the Grafana live map in real time.
- **Supervisor Review & Sign-Off:** Formal milestone review sessions conducted with Eng. Abdelrahman Mousa at the conclusion of Sprint 1 and Sprint 3 to validate design integrity.

**7. Manage Changes**

**Change-Control & Configuration Management**

- **Version Control Strategy:** All source code, Docker configurations, and SQL schemas will be tracked in a GitHub repository using branch protection on main. Each feature (feature/mqtt-simulator, feature/timescaledb-schema) requires a pull request and code review.
- **Requirements Traceability Matrix (RTM):**
    - FR-01 → Handled by Zakaria Ahmed (simulator/producer.py).
    - FR-02 & FR-03 → Handled by Abdalaziz Ayman & Youssef Elsayed (broker/ & consumer/).
    - FR-04 & FR-05 → Handled by Abdelmonem Khaled & David Fady (database/schema.sql).
    - FR-06 & FR-07 → Handled by Muhammad Ali (grafana/dashboards/).
- **Scope Change Management:** Any addition to the schema or infrastructure must be submitted as a GitHub Issue, evaluated for timeline impact, and formally approved by the Team Leader and Supervisor before implementation.